SB SECURITY

GoldMine

Security Review

March 26, 2026 - April 2, 2026

Contents

1. About SBSecurity 3

2. Disclaimer 3

3. Risk classification 3

4. Executive Summary 4

5. Findings 5

Low severity 6

[L-01] Less genesis amount when starting miner with ETH and Diamonds 6
[L-02] Sync Before Cycle Distribution Misattributes Rewards to Expired Cycle 7
[L-03] GoldMineReserve::unlock Conditionally Skipped When No Leftover GOLDMINE 8
[I-01] Missing check for miner amount 9
[I-02] Missing Early Zero Deposit Check in auction claim 10
[I-03] Reserve claim Distributes Based on Balance Instead of Claimed Amount 11

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **richland100/GOLDMINE** repository was conducted by **SBSecurity**. The review was performed by a team of 2 security researchers, who identified **6 issues** in total.

Overview

Project	GoldMine
Commit Hash	f47227ef980c5ccf280a53144597b81ecfa25fbf
Resolution	721111d222d0f71276921ba39f4603eec058f28b
Timeline	March 26, 2026 - April 2, 2026

Scope

GoldMine.sol, GoldMineAuctions.sol, GoldMineLpInjector.sol, GoldMineOracle.sol, GoldMineReserve.sol, GoldMineVault.sol

Post Audit Condition

The codebase demonstrates solid development practices overall. Additional audits will be bonus, but not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low/Info Risk	6

ID	Title	Severity	Resolution
[L-01]	Less genesis amount when starting miner with ETH and Diamonds	Low	**Fixed**
[L-02]	Sync Before Cycle Distribution Misattributes Rewards to Expired Cycle	Low	Acknowledged
[L-03]	`GoldMineReserve::unlock` Conditionally Skipped When No Leftover `GOLDMINE`	Low	**Fixed**
[I-01]	Missing check for miner amount	Info	**Fixed**
[I-02]	Missing Early Zero Deposit Check in auction claim	Info	Acknowledged
[I-03]	Reserve claim Distributes Based on Balance Instead of Claimed Amount	Info	Acknowledged

Low severity

[L-01] Less genesis amount when starting miner with ETH and Diamonds

Severity: Low Risk

Description: Miner can be started with ETH only or with ETH and Diamonds, and the token distribution should be the same in both cases. However, an incorrect percentage is used for the genesis portion of the Diamonds tokens, which results in incorrect genesis and Vault allocations.

Impact:

Lower genesis allocation and higher Vault allocation when starting Miner with ETH and Diamonds.

Proof of Concept:

When calculating the genesis portion in the ETH-only function, 8% is applied to 100% of the amount because only ETH is used.

```
uint256 wbtcGenesis = wbtcAmount * GENESIS_BPS / ETH_WBTC_LEG;
```

In the ETH and Diamonds case, 8% is first applied to 100% of the ETH portion, then the genesis from the Diamonds portion is added, with 8% applied to 50% of the Diamonds portion, resulting in half as much genesis from the Diamonds.

```
uint256 wbtcGenesis = _applyBps(wbtcAmountFromEth, GENESIS_BPS); // 8% of 100% ETH
uint256 wbtcVault = wbtcAmountFromEth - wbtcToXaut - wbtcLp - wbtcGenesis;

uint256 diamondsToWbtcAmount = diamondsAmount - diamondsBurn; // 50% Diamonds
GOLDMINE_ORACLE.diamondsWbtcTwapCheck(diamondsToWbtcAmount, minAmountWbtcFromDiamonds);
uint256 wbtcAmountFromDiamonds = _handleDiamondsSell(diamondsToWbtcAmount, minAmountWbtcFromDiamonds,
deadline);
uint256 wbtcGenesis2 = _applyBps(wbtcAmountFromDiamonds, GENESIS_BPS); // 8% of 50% Diamonds

wbtcGenesis += wbtcGenesis2;
```

Recommendation: Double the percentages so they apply to 100% of the Diamonds.

```
function startMiner(
  ...

        uint256 diamondsToWbtcAmount = diamondsAmount - diamondsBurn;
        GOLDMINE_ORACLE.diamondsWbtcTwapCheck(diamondsToWbtcAmount, minAmountWbtcFromDiamonds);
        uint256 wbtcAmountFromDiamonds = _handleDiamondsSell(diamondsToWbtcAmount,
minAmountWbtcFromDiamonds, deadline);
-       uint256 wbtcGenesis2 = _applyBps(wbtcAmountFromDiamonds, GENESIS_BPS);
+       uint256 wbtcGenesis2 = _applyBps(wbtcAmountFromDiamonds, GENESIS_BPS * 2);

  ...
```

Resolution: Fixed

[L-02] Sync Before Cycle Distribution Misattributes Rewards to Expired Cycle

Severity: Low Risk

Description: In `update`, `_syncIncomingRewards` runs before the cycle distribution logic. Since `update` only executes when a user interacts with the contract via the `autoUpdate` modifier, there will almost always be a delay between when a cycle boundary passes and when `update` is actually called. Any WBTC that arrives at the contract during this gap gets synced into `pool88Day` and `pool288Day` first, then immediately distributed as part of the expired cycle. These rewards should belong to the next cycle instead.

Impact:

Stakers active during the expiring cycle receive rewards that arrived after the cycle ended. The next cycle starts with a smaller reward pool than it should. This is not an edge case but the expected execution path, as the delay between a cycle boundary and the next `update` call is variable and depends entirely on user activity.

Proof of Concept:

1. The 88-day cycle ends at timestamp T

2. 10 WBTC is sent to the vault between T and T + 3 days

3. A user calls `deposit`, `withdraw`, or `claimRewards` at T + 3 days, triggering `autoUpdate`

4. `_syncIncomingRewards` runs first, adding 6 WBTC to `pool88Day` and 4 WBTC to `pool288Day`

5. The cycle boundary check passes and distributes the entire `pool88Day`, including the 6 WBTC that arrived after the cycle ended

6. Those 6 WBTC should have been allocated to the next 88-day cycle

```
function update() public {
    _syncIncomingRewards(); // @audit syncs new WBTC before cycle check
    if (totalShares == 0) return;
    if (block.timestamp >= last88dayPayout + POOL_88_LENGTH) {
        uint256 perSharePayout = _calculatePerShareValue(pool88Day);
        _totalWbtcPerShare += perSharePayout;
        pool88Day -= _convertShareValue(perSharePayout, totalShares);
        last88dayPayout = _getCurrentCycleStart(POOL_88_LENGTH);
    }
    if (block.timestamp >= last288dayPayout + POOL_288_LENGTH) {
        uint256 perSharePayout = _calculatePerShareValue(pool288Day);
        _totalWbtcPerShare += perSharePayout;
        pool288Day -= _convertShareValue(perSharePayout, totalShares);
        last288dayPayout = _getCurrentCycleStart(POOL_288_LENGTH);
    }
}
```

Recommendation: Move `_syncIncomingRewards` after the cycle distribution logic. Keep it before the early return for the `totalShares == 0` case so rewards still accumulate in the pools when there are no stakers.

Resolution: Acknowledged

[L-03] `GoldMineReserve::unlock` Conditionally Skipped When No Leftover `GOLDMINE`

Severity: Low Risk

Description: `GoldMineReserve` uses an unlocked flag to gate the `stake` function. When a user calls `stake`, it sets `unlocked` to false. The only way to re-enable staking is through `unlock`, which can only be called by the LP Injector.

In `inject`, the call to unlock is inside a conditional block that only executes when there is leftover `GOLDMINE` after adding liquidity:

```
uint256 leftoverGoldmine = goldmineBalance - usedGoldmine;
uint256 reserveAmount;
if (leftoverGoldmine > 0) {
    if (reserveFundingEnabled) {
        reserveAmount = leftoverGoldmine / 2;
        goldmine.safeTransfer(GOLDMINE_RESERVE, reserveAmount);
        leftoverGoldmine -= reserveAmount;
    }
    goldmine.burn(leftoverGoldmine);
    IGoldMineReserve(GOLDMINE_RESERVE).unlock();
}
```

When Uniswap consumes 100% of the provided `GOLDMINE (leftoverGoldmine == 0)`, unlock is never called. The Reserve stays locked and any `GOLDMINE` in it cannot be staked. This can persist across multiple consecutive weeks if the pool ratio keeps absorbing all `GOLDMINE`.

Impact:

`GOLDMINE` accumulates in `GoldMineReserve` without being staked into the vault, forgoing yield for an indefinite period. The stake function reverts with Cooldown on every call, making the staking incentive mechanism non-functional. If reserveFundingEnabled is true, the Reserve also misses receiving its share of `GOLDMINE` during these weeks, compounding the idle balance.

Recommendation: Move unlock outside the conditional block:

```
uint256 leftoverGoldmine = goldmineBalance - usedGoldmine;
uint256 reserveAmount;
if (leftoverGoldmine > 0) {
    if (reserveFundingEnabled) {
        reserveAmount = leftoverGoldmine / 2;
        goldmine.safeTransfer(GOLDMINE_RESERVE, reserveAmount);
        leftoverGoldmine -= reserveAmount;
    }
    goldmine.burn(leftoverGoldmine);
}
IGoldMineReserve(GOLDMINE_RESERVE).unlock();
```

Resolution: Fixed

[I-01] Missing check for miner amount

Severity: Information Risk

Description: When starting a miner using the only Eth function, 50% of the ETH is virtually treated as Diamonds to calculate how much the miner will mint. The main ETH-and-Diamonds miner function includes a minimum check for the miner amount, but the only Eth function is missing this check.

```
function startMiner(uint256 minAmountDiamonds, uint256 minAmountWbtc, uint256 minAmountXaut, uint256
deadline)
    external
    payable
{
    if (block.timestamp < PROTOCOL_START) revert MiningInactive();
    uint256 ethAmount = msg.value;
    if (ethAmount == 0) revert ZeroInput();
    IWETH9(WETH).deposit{value: ethAmount}();
    uint256 ethToDiamondsAmount = _applyBps(ethAmount, ETH_DIAMONDS_BURN);
    GOLDMINE_ORACLE.ethDiamondsTwapCheck(ethToDiamondsAmount, minAmountDiamonds);
    uint256 diamondsBurn = _handleDiamondsBuy(ethToDiamondsAmount, minAmountDiamonds, deadline);
    // Extrapolate to 50% of eth swapper
    uint256 minerAmount = diamondsBurn * BPS_BASE / DIAMONDS_BURN_BPS; <-------
```

```
function startMiner(
    uint256 diamondsAmount,
    uint256 minAmountWbtc,
    uint256 minAmountXaut,
    uint256 minAmountWbtcFromDiamonds,
    uint256 deadline
) external payable {
    if (block.timestamp < PROTOCOL_START) revert MiningInactive();
    if (diamondsAmount < MIN_MINER_AMOUNT) revert InsufficientDiamonds(); <---------
    uint256 ethAmount = msg.value;

    IERC20(DIAMONDS).safeTransferFrom(msg.sender, address(this), diamondsAmount);
    GOLDMINE_ORACLE.diamondsPriceCheck(ethAmount, diamondsAmount);

    uint256 minerId = _writeMinerData(diamondsAmount);
```

Recommendation: Add the check to the ETH-only function as well.

Resolution: Fixed

[I-02] Missing Early Zero Deposit Check in auction claim

Severity: Information Risk

Description: In `claim`, `_calculateAuctionPayout` is called before checking whether the user has any deposit for the given week. If `depositOf[user][auctionWeek]` is 0, the function still reads `weeklyStats[auctionWeek]` from storage and performs a multiplication before ultimately returning 0, which then triggers the `NothingToClaim` revert.

Impact:

Users with no deposit for a given week waste gas on an unnecessary storage read and computation before reverting.

Recommendation: Add an early zero check before calling `_calculateAuctionPayout`:

```
+ uint256 userDeposit = depositOf[user][auctionWeek];
+ if (userDeposit == 0) revert NothingToClaim();
- uint256 payout = _calculateAuctionPayout(depositOf[user][auctionWeek], auctionWeek);
+ uint256 payout = _calculateAuctionPayout(userDeposit, auctionWeek);
  if (payout == 0) revert NothingToClaim();
```

Resolution: Acknowledged

[I-03] Reserve claim Distributes Based on Balance Instead of Claimed Amount

Severity: Information Risk

Description: In `claim`, `incentive` is calculated from `claimAmount` (the vault rewards just claimed), but `distributionAmount` is derived from the contract's full `balanceOf` minus `incentive`. If the contract holds any residual WBTC from prior claims or direct transfers, that extra balance gets forwarded to the vault as well.

```
uint256 incentive = _applyBps(claimAmount, claimIncentiveFeeBps);
uint256 distributionAmount = wbtc.balanceOf(address(this)) - incentive;

wbtc.safeTransfer(GOLDMINE_VAULT, distributionAmount);
```

Impact:

Any residual WBTC in the contract is swept to the vault without an incentive fee applied to it. In normal operation the balance should equal `claimAmount`, so this is a consistency concern rather than a fund loss risk.

Recommendation: Compute `distributionAmount` directly from `claimAmount`:

```
- uint256 distributionAmount = wbtc.balanceOf(address(this)) - incentive;
+ uint256 distributionAmount = claimAmount - incentive;
```

Or base the incentive on the WBTC balance as well.

Resolution: Acknowledged